



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”

Campus Presidente Prudente

CÁLCULO CIENTÍFICO E APRENDIZADO DE MÁQUINA:
DA TEORIA À APLICAÇÃO

Relatório Final
Bolsa de Iniciação Científica - PIBIC

Aluno: Henrique Dantas Quintana
Orientadora: Analice Costacurta Brandi

`henrique.quintana@unesp.br`

`analice.brandi@unesp.br`

NOVEMBRO DE 2024
PRESIDENTE PRUDENTE-SP

Resumo. *O aprendizado de máquina é uma área da Inteligência Artificial que se concentra no desenvolvimento de modelos e algoritmos capazes de aprender dados sem serem explicitamente programados; esses algoritmos visam identificar padrões, fazer previsões e tomar decisões usando experiências, exemplos anteriores, observações, recomendações ou reforços. Exemplos de técnicas de aprendizado de máquina incluem redes neurais, árvores de decisão e algoritmos de agrupamento. Por outro lado, o cálculo científico envolve a aplicação de métodos matemáticos e computacionais para resolver problemas científicos, especialmente aqueles que requerem análise e simulação numérica.*

Palavras Chaves: *aprendizado de máquina; redes neurais; Cálculo científico; difusão de calor.*

Abstract. *Machine learning is a field of Artificial Intelligence focused on developing models and algorithms that can learn from data without being explicitly programmed. These algorithms aim to identify patterns, make predictions, and make decisions based on experiences, previous examples, observations, recommendations, or reinforcement. Examples of machine learning techniques include neural networks, decision trees, and clustering algorithms. On the other hand, scientific computing involves the application of mathematical and computational methods to solve scientific problems, particularly those that require analysis and numerical simulation.*

Keywords: *Machine learning; Neural Network; Machine Learning; Scientific Calculus; heat diffusion.*

1. INTRODUÇÃO

O aprendizado de máquina (ML) é uma subárea da inteligência artificial que se dedica ao desenvolvimento de algoritmos capazes de aprender e realizar tarefas preditivas com base em dados, sem programação explícita. A aplicação de técnicas de ML tem se expandido rapidamente em diversas áreas do conhecimento, destacando-se por sua capacidade de identificar padrões complexos, realizar previsões e automatizar processos em diferentes contextos.

Por outro lado, o cálculo científico é uma disciplina que se dedica à aplicação de métodos matemáticos e computacionais para resolver problemas científicos e de engenharia. Um dos desafios dessa área é a resolução de equações diferenciais parciais (EDPs), que frequentemente modelam fenômenos físicos como a difusão de calor, propagação de ondas, etc. A solução dessas equações são essenciais para entender o comportamento de sistemas complexos.

No entanto, a combinação entre aprendizado de máquina e métodos de cálculo científico tem demonstrado um potencial significativo para resolver problemas numéricos e simulações em campos como física e engenharia.

Este projeto de Iniciação Científica busca explorar como essas duas áreas podem ser utilizadas, com o objetivo de criar novas estratégias para resolver problemas reais. Ao integrar o conhecimento do aprendizado de máquina com o cálculo científico, é possível explorar todo o potencial dessas áreas interdisciplinares e promover avanços significativos.

Este relatório explora a resolução da equação de difusão de calor unidimensional utilizando redes neurais, investigando a eficácia do método. A difusão de calor, um fenômeno físico descrito por equações diferenciais parciais (EDPs), serve como um estudo de caso para demonstrar como redes neurais físicas informadas (PINNs) podem ser aplicadas para obter soluções precisas. O projeto inclui a implementação de uma rede neural treinada para prever o comportamento da solução da EDP e discute a viabilidade dessa abordagem em termos de precisão e eficiência.

2. MATERIAIS E MÉTODOS

Esta seção é dedicada à discussão sobre os conceitos básicos relacionados aos temas da difusão de calor, cálculo científico, Machine Learning e ferramentas de análise para os resultados.

2.1. MACHINE LEARNING

Machine Learning (ML), uma categoria pertencente à área de IA tem como objetivo “treinar” máquinas com intenção de resolver problemas, automatizar tarefas e identificar padrões.

Com as capacidades modernas da *Machine Learning* aumentando, a área deixou de apenas ser estruturação e dedução de dados, evoluindo para entender padrões indefinidos usando técnicas de treinamento do qual cada padrão aprendido utilizado, pode ser utilizado para analisar, outros padrões indefinidos, compreender dados e automatizar tarefas. Hoje em dia, com a capacidade computacional da nuvem e a abrangente quantidade de dados disponíveis, ou até mesmo gerados por outras ML para treinar MLs, demonstrando a capacidade de ser aplicada em diversos cenários diferentes para chegar a um resultado

esperado/procurado, ajudando em várias áreas de atuações nas profissões, com as maiores contribuições da ML sendo para as áreas da saúde e áreas de processamento de imagens, por exemplo. Este trabalho por sua vez foca em trazer essas ideias de aplicações de ML para área do cálculo científico, mais especificamente na aplicação de problemas.

O aprendizado consiste na modificação dos pesos das conexões entre os neurônios, os pesos iniciais (sinapses) são modificados de forma iterativa, por um algoritmo que podem seguir por um dos seguintes paradigmas de aprendizado:

- **Aprendizado Supervisionado:** é apresentado um conjunto de treino, consistindo de entradas e correspondentes saídas desejadas.
- **Aprendizado por Reforço:** para cada entrada apresentada, é produzida uma indicação (reforço) sobre a adequação das saídas correspondentes produzidas pela rede.
- **Aprendizado não-supervisionado:** a rede atualiza seus pesos sem o uso de pares entrada-saídas desejadas e sem indicações sobre a adequação das saídas produzidas.

O paradigma de aprendizado que foi utilizado para a realização do trabalho será o Aprendizado Supervisionado.

2.2 REGRESSÃO LINEAR

Para se entender outros conceitos mais complexos de aprendizado de máquina, é necessário saber dos conceitos mais simples.

Regressão linear é uma área do aprendizado de máquina com paradigma que visa descobrir uma reta que consiga definir bem os dados de uma amostra.

A hipótese é uma função $\hat{y} = b + mx$ que tenta descrever todos os pontos de uma amostra, onde b é onde a reta toca o eixo $y = 0$ e m é a inclinação da reta, renomeando $a = w_1$ e $b = w_0$, $\hat{y} = w_0 + w_1 x$. O erro da hipótese é $(\hat{y} - y)^2$, onde $\hat{y}$ é o valor predito e y o valor original. O custo é a somatória $\frac{1}{2} \sum_{i=1}^m \frac{(\hat{y}(i) - y(i))^2}{m}$, chamada de média do erro ao quadrado (MSE), de todos os erros da hipótese dos pontos de amostra, quanto menor o custo, significa que a hipótese irá acertar com mais precisão, logo descobre uma configuração otimizada.

A função de custo pode ser vista como uma função quadrática positiva, a melhor

otimização pode ser feita através da derivada da função no ponto de mínimo, onde a tangente é aproximadamente zero, para isso, é utilizado o método gradiente descendente, o gradiente ∇ é a direção de como que um ponto se chega no ponto de mínimo da função e α é a taxa de aprendizagem. Um algoritmo é repetido n vezes: $w_0 = w_0 - \alpha \nabla_{w_0}$ e $w_1 = w_1 - \alpha \nabla_{w_1}$, sempre atualizando os valores w_0 e w_1 , quando n aumenta, um ponto qualquer vai se aproximar do ponto de mínimo da função de custo, o custo diminui e a reta que descreve os pontos da amostra fique com uma configuração mais otimizada.

O gradiente ∇_{w_0} e ∇_{w_1} são calculadas através da derivada da função de custo em relação a w_0 e w_1 , respectivamente, então $\nabla_{w_0} = \frac{1}{m} \sum_{i=1}^m (\hat{y} - y)$ e $\nabla_{w_1} = \frac{1}{m} \sum_{i=1}^m (\hat{y} - y)x$.

O algoritmo final fica: fazer n vezes, faça: $w_0 = w_0 - \alpha \frac{1}{m} \sum_{i=1}^m erro$

$$w_1 = w_1 - \alpha \frac{1}{m} \sum_{i=1}^m erro \cdot x$$

Esse algoritmo mostra como uma máquina aprende, alterando os pesos em cada iteração, diminuindo o erro do algoritmo, indo cada vez mais para uma solução melhor aproximada.

2.3 NEURÔNIOS

Um neurônio de uma rede neural é um componente que calcula a soma ponderada de vários *inputs*, aplica uma função e passa o resultado adiante.

No modelo perceptron, cada sinal de entrada está associado ao seu respectivo peso. É construída a partir do vetor de *input* e o vetor de *weights* uma combinação linear $z = w_0 * 1 + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$.

A função de ativação então pega essa combinação linear e aplica uma transformação não linear a ela. Essa transformação é o que permite que redes neurais aprendam relações complexas e não lineares entre os dados de entrada e saída.

O input $w_0 = 1$ é o *bias*. *Bias* em redes neurais é um valor adicional que é adicionado a cada neurônio, juntamente com as entradas, antes de ser aplicada uma função de ativação. Esse valor de bias permite que a rede neural faça ajustes na função de ativação, deslocando-a para cima ou para baixo. Em outras palavras, o bias permite que a rede neural aprenda a melhor forma de mapear as entradas para as saídas desejadas. Sem o bias, a função de ativação seria

sempre centrada em zero, o que limitaria a capacidade da rede neural de aprender padrões complexos e não-linearidades nos dados.

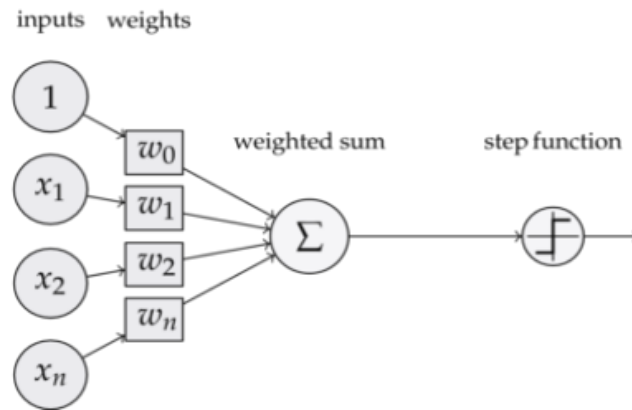


Figura 1 - Neurônio.

2.4 REDES NEURAIS

Quando utilizamos vários neurônios em paralelo temos uma rede neural. Cada neurônio recebe os sinais das variáveis dos *inputs* e passando adiante uma versão ponderada e tratada a partir de uma função de ativação desse sinal. Esses neurônios em paralelo formam uma camada oculta da rede neural. Pode-se tratar o *output* de cada neurônio como uma variável do *input* de uma outra camada. Quando uma rede neural possui várias camadas ocultas, é chamada de rede neural profunda (DNN - Deep Neural Network).

Uma rede densa significa que todos os neurônios de uma camada estão conectados a todos os neurônios da camada anterior, tem-se então uma camada totalmente conectada.

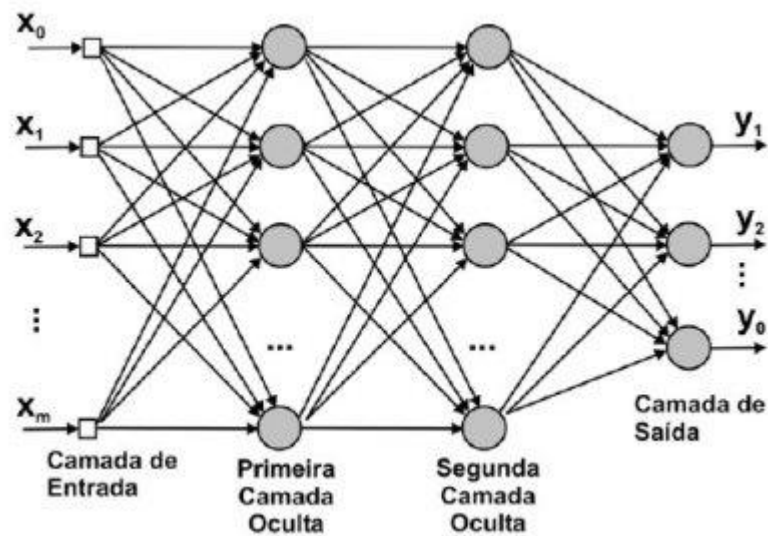


Figura 2 - Exemplo de rede densa.

2.5 BACKPROPAGATION

Backpropagation, ou retropropagação, é um algoritmo que ajusta os pesos de uma rede neural artificial para que ela aprenda a partir de dados e minimize o erro entre a saída desejada e a obtida. É um método padrão para treinar redes neurais e é fundamental para que elas melhorem seu desempenho ao longo do tempo.

O algoritmo funciona em duas fases:

- **Feed-Foward:** os dados de entrada são passados pela rede neural, camada por camada, até que se obtenha uma saída. Durante essa etapa, os pesos das conexões são mantidos fixos.
- **Feed-Backward:** após a obtenção da saída, é calculado o erro entre a saída obtida e a saída desejada. Esse erro é então utilizado na etapa de propagação para trás, onde os pesos das conexões são ajustados de forma a minimizar esse erro.

O backpropagation é basicamente uma aplicação da regra da cadeia, um princípio matemático que permite calcular a derivada de uma função composta, mais especificamente, a derivada da função de erro do neurônio de saída, que, a partir da derivada, é possível alterar os pesos das arestas e dos bias, durante a execução do programa nas redes neurais.

No Backpropagation, a função de ativação é utilizada tanto na propagação para frente, para calcular a saída de cada neurônio, quanto na propagação para trás, para calcular o gradiente descendente. A escolha da função de ativação pode influenciar no desempenho da

rede neural, uma vez que diferentes funções possuem diferentes características.

Foi utilizado nesse projeto a função de ativação tangente hiperbólica ($\tanh$) para as redes ocultas, pois a função $\tanh$ gera valores no intervalo de -1 a +1. Isso significa que pode lidar com valores negativos de forma mais eficaz do que as outras funções. A $\tanh$ é centrada em zero, o que significa que sua saída é simétrica em torno da origem do sistema de coordenadas. Isso geralmente é considerado uma vantagem, pois pode ajudar o algoritmo de aprendizado a convergir mais rapidamente.

2.6 PINN

Concederemos que recebemos alguns pontos de dados experimentais que vêm de algum fenômeno físico desconhecido.

Uma tarefa científica comum é encontrar um *modelo* que seja capaz de prever com precisão novas medições experimentais a partir desses dados.

Uma maneira popular de fazer isso usando machine learning é usar uma rede neural. Dada a localização de um ponto de dados como entrada (denotado x), uma rede neural pode ser usada para emitir uma previsão de seu valor (denotado u), conforme mostrado na figura abaixo:

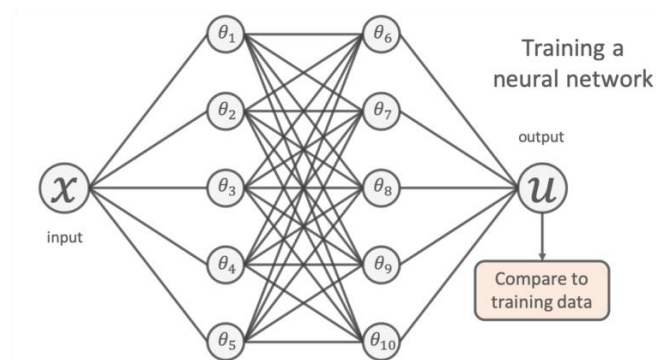


Figura 3 - Rede PINN.

O problema de usar uma abordagem puramente orientada a dados como essa pode ter desvantagens significativas, pois, embora a rede neural modele com precisão o processo físico dentro da vizinhança dos dados experimentais, ela falha em *generalizar* para longe desses dados de treinamento. Ao confiar apenas nos dados, pode-se argumentar que ela não "entendeu" verdadeiramente o problema científico.

Para contornar isso, a rede neural informada pela física (PINN) é capaz de prever a

solução bem longe dos pontos de dados experimentais, pois, além de ter o custo sobre os dados experimentais, é possível adicionar as equações diferenciais conhecidas diretamente na função de custo total ao treinar a rede neural.

Isso é feito mostrando um conjunto de locais de treinamento de entrada e passando-os pela rede. Os próximos gradientes da saída da rede com relação à sua entrada são computados na rede. Finalmente, o resíduo da equação diferencial subjacente é computado usando esses gradientes e adicionado como um termo extra na função de perda.

2.7 EQUAÇÕES DIFERENCIAIS

Muitos dos princípios, ou leis, que regem o comportamento do mundo físico são proposições, ou relações, envolvendo a taxa segundo a qual as coisas acontecem. Expressas em linguagem matemática, as relações são equações e as taxas são derivadas. Equações contendo derivadas são equações diferenciais. Portanto, para compreender e investigar problemas envolvendo o movimento de fluidos, o fluxo de corrente elétrica e circuitos, entre muitos outros, é necessário saber alguma coisa sobre equações diferenciais.

Uma equação que contém as derivadas ou diferenciais de uma ou mais variáveis dependentes, em relação a uma ou mais variáveis independentes, é chamada de equação diferencial (ED).

Equações diferenciais são classificadas de acordo com o tipo, a ordem e a linearidade.

2.7.1. TIPOS DE EQUAÇÕES DIFERENCIAIS

As EDs podem ser classificadas por Ordinária ou Parcial.

2.7.1.1. Equação Diferencial Ordinária (EDO)

Se uma equação contém somente derivadas ordinárias de uma ou mais variáveis dependentes, com relação a uma única variável independente, é chamada de equação diferencial ordinária.

Exemplos: $\frac{dy}{dt} + 2y = 1$

$$\frac{du}{dt} + \frac{dv}{dt} = x$$

2.7.1.2. Equação Diferencial Parcial (EDP)

Uma equação que envolve as derivadas parciais de uma ou mais variáveis dependentes de duas ou mais variáveis independentes é chamada de equação diferencial parcial.

Exemplos: $\frac{\partial u}{\partial t}(x, t) = \alpha^2 \frac{\partial^2 u}{\partial x^2}(x, t)$ (equação de calor)

$$\frac{\partial u}{\partial y} = -\frac{\partial v}{\partial x}$$

2.7.2. ORDEM DE EQUAÇÕES DIFERENCIAIS

A ordem de uma equação diferencial é a ordem da mais alta derivada que está na equação.

$$4t \frac{\partial u}{\partial t} + 5t = 20u \quad (\text{EDO de primeira ordem})$$

$$u \frac{\partial^2 u}{\partial t^2} + 5t = 6t \frac{\partial^3 v}{\partial t^3} \quad (\text{EDP de terceira ordem})$$

2.8 SÉRIE DE FOURIER EM SENOS

A série de Fourier é uma ferramenta matemática usada para decompor funções periódicas em uma soma infinita de funções senos e cossenos. Essa técnica foi desenvolvida por Jean-Baptiste Joseph Fourier no início do século XIX e é fundamental em áreas como física, engenharia e processamento de sinais. A série é expressa da seguinte forma: $f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right]$, onde a_0 é o primeiro termo da série e a_n e b_n são coeficientes de Fourier. A série de Fourier é uma série trigonométrica quando converge.

Os coeficientes a_n e b_n são calculados pela integração da função $f(x)$ multiplicada por senos e cossenos. A fórmula para calcular cada coeficiente é: $a_n = \frac{1}{L} \int_{-L}^L f(x) \cos\left(\frac{n\pi x}{L}\right) dx$ e $b_n = \frac{1}{L} \int_{-L}^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx$.

A série de Fourier de uma função $f(x)$ que é ímpar, assume a seguinte forma: $f(x) = \sum_{n=1}^{\infty} b_n \sin\left(\frac{n\pi x}{L}\right)$, pois $\frac{1}{L} \int_{-L}^0 f(x) \cos\left(\frac{n\pi x}{L}\right) dx = -\frac{1}{L} \int_0^L f(x) \cos\left(\frac{n\pi x}{L}\right) dx$, logo $a_n = \frac{1}{L} \int_{-L}^L f(x) \cos\left(\frac{n\pi x}{L}\right) dx = 0$ e $b_n = \frac{1}{L} \int_{-L}^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx = \frac{2}{L} \int_0^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx$.

Portanto, a série de Fourier de uma função ímpar é formada apenas pelas funções trigonométricas ímpares da forma $\sin\left(\frac{n\pi x}{L}\right)$. Esse tipo de série é chamado de série de Fourier

em senos.

2.9 MATPLOTLIB, NUMPY

Numpy é uma biblioteca de Python criada utilizando vetorização invés de loops, arrays e indexação, utilizando código de máquina pré compilado em C baseada no objeto *array* da biblioteca para realizar as operações com velocidade e consistência, pois todas as operações necessárias estão pré-compiladas e armazenadas pronto para serem utilizadas.

Matplotlib⁷ é uma biblioteca implementada em Python para visualização de dados em forma gráfica, permitindo visualização de dados conforme a necessidade do programador ao realizar testes, experimentos, pesquisas e trabalhar com resultados obtidos de projetos.

2.10 TENSORFLOW, KERAS

TensorFlow, é um framework de algoritmos de *Deep Learning* e *Machine Learning*, desenvolvido pela *Google Brains* e tornado *open-source* em 2015, sendo uma biblioteca aberta utilizando computação numérica em larga escala.

O *TensorFlow* cria grafos/estruturas permitindo o fluxo destes sejam processados, cada vértice do grafo possui a representatividade de operações matemáticas e cada aresta, um array multidimensional, chamado de *tensor*.

A biblioteca permite funcionar localmente, consumindo GPU e CPU da máquina do programador. No desenvolvimento do projeto, no entanto, foi utilizado CPU e a GPU da Google colab, plataforma online gratuita do Google que permite a execução de código Python no navegador, com acesso gratuito GPUs dada pela própria Google.

A biblioteca escolhida para a implementação deste trabalho foi Keras pela sua simplicidade para desenvolver modelos. Keras é uma biblioteca de rede neural de alto nível de código aberto, escrita em Python e capaz de rodar sobre TensorFlow, Microsoft Cognitive Toolkit, Theano ou PlaidML, bibliotecas de código aberto para aprendizado de máquina aplicáveis a uma ampla variedade de tarefas.

3. RESULTADOS E DISCUSSÃO

Neste trabalho, a solução numérica de uma equação diferencial parcial (EDP) foi obtida por meio de métodos de discretização, essa solução é necessária para desenhar os gráficos das soluções numéricas com valores diferentes de α e tempo, e para poder comparar com a predição da rede neural física informada (PINN) que é calculada durante

a mudança de estados da rede. A saída da rede representa a solução da equação.

Para a construção da rede neural, utilizou-se uma arquitetura de duas camadas ocultas, com funções de ativação tangente hiperbólica e a função de ativação da saída com função linear. A EDP foi utilizada para fazer o treinamento da rede. O treinamento foi realizado usando algoritmos de otimização para ajustar os pesos da rede, buscando convergência para a solução numérica da EDP.

O problema em questão envolve uma equação diferencial parcial (EDP) parabólica unidimensional dada por $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$, em que t é a variável temporal, x é a variável espacial e α uma constante.

Considerando a equação de difusão de calor unidimensional dada por $\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}$, em que T é a temperatura e α é o coeficiente de difusividade térmica do material. Considere uma barra delgada de comprimento $L = 1m$, a uma temperatura inicial $T_{init} = 0^\circ C$. A barra possui sua extremidade direita colocada à temperatura $T_n = 100^\circ C$, enquanto que a extremidade esquerda é mantida em $T_o = 0^\circ C$, portanto, para descobrir a solução analítica através da EDP, o problema pode ser descrito por um problema de valor inicial e de fronteira (PVIF) de fronteiras não homogêneas, ou seja, T_n e T_o possuem valores diferentes, dada por

$$\begin{cases} \frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \\ T(x, 0) = f(x), 0 < x < 1 \\ T(0, t) = 0, T(L, t) = 100 \end{cases}$$

onde $f(x)$ é a condição inicial do problema, ou seja, $f(x) = T_{init} = 0$.

Depois de certo tempo ($t \rightarrow \infty$), na barra inteira será alcançada uma temperatura estacionária $v(x)$, que independe do tempo e das condições iniciais, portanto, a derivada $v'(t)$ em relação a t é 0, $\alpha \frac{\partial^2 T}{\partial x^2} = 0 \Rightarrow \frac{\partial^2 T}{\partial x^2} = 0$ então a segunda derivada $v''(x)$ em relação a x é 0, como a derivada de primeira e segunda ordem são 0 a função v é da forma $v(x) = aX + b$, onde $v(0) = 0$ e $v(L) = v(1) = 100$, pois $T(0, t) = 0$ e $T(1, t) = 100$, respectivamente. Assim, $v(0) = T_o = b$ e $v(L) = aL + T_o = T_n \Rightarrow a = \frac{T_n - T_o}{L}$.

Portanto $v(x) = \frac{T_n - T_o}{L} x + T_o$, substituindo os valores, tem-se $v(x) = 100x$.

Expressando $T(x, t)$ como $v(x) + w(x, t)$, derivando T em relação a t e derivando

em segunda ordem T em relação a x, tem-se que $\frac{\partial T}{\partial t} = \frac{\partial v}{\partial t} + \frac{\partial w}{\partial t}$ e $\frac{\partial^2 T}{\partial x^2} = \frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 w}{\partial x^2}$ respectivamente, porém, como $\frac{\partial^2 v}{\partial t^2} = 0$ e $\frac{\partial v}{\partial t} = 0$, então $\frac{\partial T}{\partial t} = \frac{\partial w}{\partial t}$ e $\frac{\partial^2 T}{\partial x^2} = \frac{\partial^2 w}{\partial x^2}$, substituindo da EDP $\frac{\partial w}{\partial t} = \alpha \frac{\partial^2 w}{\partial x^2}$, com novas condições de fronteira:

$$1. w(0,t) = T(0,t) - v(0) \Rightarrow w(0,t) = 0 - T_0 = 0;$$

$$2. w(L,t) = T(L,t) - v(L) \Rightarrow 100 - 100 = 0;$$

como $T(x,t) = v(x) + w(x,t)$, $T(x,0) = v(x) + w(x,0) \Rightarrow w(x,0) = T(x,0) - v(x) \Rightarrow w(x,0) = f(x) - v(x)$, criando assim uma nova EDP descrita por uma outra PVIF, porém de fronteiras homogêneas, ou seja, as duas extremidades possuem valores iguais.

$$\begin{cases} \frac{\partial w}{\partial t} = \alpha \frac{\partial^2 w}{\partial x^2} \\ w(x,0) = f(x) - v(x), 0 < x < 1 \\ w(0,t) = w(L,t) = 0, L = 1 \end{cases}$$

Existe um método para procurar essa solução, chamado de separação de variáveis, que consiste em procurar uma função X que só depende da posição x e uma função T que só depende do tempo t, multiplicando as duas funções de modo que essa seja a solução procurada. Sendo assim $w(x,t) = X(x) \cdot T(t)$.

Será necessário descobrir o $\frac{\partial w}{\partial t}$ e $\frac{\partial^2 w}{\partial x^2}$, então derivando w em relação a t: $\frac{\partial w(x,t)}{\partial t} =$

$X(x) \cdot \frac{\partial T(t)}{\partial t}$ e a derivação de segunda ordem em relação a x:

$$\frac{\partial^2 w(x,t)}{\partial x^2} = \frac{\partial^2 X(x)}{\partial x^2} \cdot T(t), \text{ substituindo na EDP: } X(x) \cdot \frac{\partial T(t)}{\partial t} = \alpha \frac{\partial^2 X(x)}{\partial x^2} \cdot T(t) \Rightarrow$$

$X(x) \cdot T'(t) = \alpha X''(x) \cdot T(t) \Rightarrow \frac{X}{X''}(x) = \frac{T}{T'}(t)$, como do primeiro lado da equação tem uma função que só depende da variável x e do outro lado outra função mas que só depende de t, essa função só é possível se for igualada a uma constante λ , construindo um sistema de equações diferenciais ordinárias (EDO).

$$\begin{cases} X''(x) = \lambda X(x) \\ T'(t) = \lambda \alpha T(t) \end{cases}$$

As condições de fronteira $w(0,t) = w(L,t) = 0$ levam a:

$$1. w(0,t) = X(0)T(t) = 0 \Rightarrow X(0) = 0$$

$$2. w(L,t) = X(L)T(t) = 0 \Rightarrow X(L) = 0$$

Supondo que a solução $X(x)$ é igual a e^{rx} , e $X'(x) = \frac{de^{rx}}{dx} = re^{rx}$ e $X''(x) = \frac{de^{rx}}{dx^2} = r^2 e^{rx}$, temos que a equação $X''(x) - \lambda X(x) = 0$, uma ED de segunda ordem e linear, fica $r^2 e^{rx} - \lambda r e^{rx} = 0 \Rightarrow e^{rx}(r^2 - \lambda) = 0 \Rightarrow r^2 - \lambda = 0$ e é a sua equação característica, que vai depender de λ . Tem-se três casos para o λ , quando $\lambda > 0$, $\lambda = 0$ e $\lambda < 0$.

- $\lambda > 0$: $r = \pm\sqrt{\lambda}$, r tem duas raízes reais como raízes reais da equação característica, então a solução para a função característica pode ser $X(x) = c_1 \cdot e^{r_1 x} + c_2 \cdot e^{r_2 x} \Rightarrow X(x) = c_1 \cdot e^{\sqrt{\lambda}x} + c_2 \cdot e^{-\sqrt{\lambda}x}$.
- $\lambda = 0$: $r = 0$, então a solução vai ficar $X(x) = c_1 \cdot e^{r_1 x} + c_2 \cdot x e^{r_2 x} \Rightarrow X(x) = c_1 \cdot e^{0x} + c_2 \cdot x \cdot e^{0x} \Rightarrow X(x) = c_1 + c_2 \cdot x$.
- $\lambda < 0$: $r = \pm i\sqrt{-\lambda}$, então $X(x) = c_1 \cdot \sin(\sqrt{-\lambda} x) + c_2 \cdot \cos(\sqrt{-\lambda} x)$.

Para o $\lambda > 0$, $X(0) = 0$, $X(L) = 0$ e $X(x) = c_1 \cdot e^{\sqrt{\lambda}x} + c_2 \cdot e^{-\sqrt{\lambda}x}$, aplicando $X(x) = 0$, temos $X(0) = c_1 \cdot e^{\sqrt{\lambda}0} + c_2 \cdot e^{-\sqrt{\lambda}0} = 0 \Rightarrow c_1 + c_2 = 0 \Rightarrow c_1 = -c_2$, portanto $X(x) = -c_2 \cdot e^{\sqrt{\lambda}x} + c_2 \cdot e^{-\sqrt{\lambda}x}$, usando agora o $X(L) = 0$, temos $X(L) = -c_2 \cdot e^{\sqrt{\lambda}L} + c_2 \cdot e^{-\sqrt{\lambda}L} = 0 \Rightarrow e^{\sqrt{\lambda}L} = e^{-\sqrt{\lambda}L} \Rightarrow \sqrt{\lambda}L = -\sqrt{\lambda}L \Rightarrow \sqrt{\lambda} = -\sqrt{\lambda}$, isso só é possível se $\lambda = 0$. Como partimos de $\lambda > 0$ e o resultado foi $\lambda = 0$, então o caso λ positivo será descartado.

Para o $\lambda = 0$, temos $X(x) = c_1 + c_2 x$ e aplicando $X(0) = 0$ se dá a função $X(0) = c_1 = 0$ e com $X(L) = 0$ se dá a função $X(L) = c_1 + c_2 L = 0 \Rightarrow c_2 L = 0 \Rightarrow c_2 = 0$. Se $c_2 = 0$, então $X(x) = 0$, logo $w(x,t) = X(x) \cdot T(t) = 0$, o que não resolve o problema.

Para o $\lambda < 0$, temos a função $X(x) = c_1 \cdot \sin(\sqrt{-\lambda} x) + c_2 \cdot \cos(\sqrt{-\lambda} x)$, aplicando $X(0) = 0$ se dá a equação $X(0) = c_1 \sin(0) + c_2 \cos(0) = 0$, obtendo $c_2 = 0$, substituindo na original, $X(x) = c_1 \cdot \sin(\sqrt{-\lambda} x)$ e aplicando $X(L) = 0$, $X(L) = c_1 \cdot \sin(\sqrt{-\lambda} L) = 0$.

Assim, temos que $c_1 \sin(\sqrt{-\lambda} L) = 0$. Se $c_1 \neq 0$, então $\sin(\sqrt{-\lambda} L) = 0$, isso só é possível quando $\sqrt{-\lambda} L$ é um múltiplo de π , ou seja, πn , para $n = 1, 2, 3, \dots$

Portanto, as condições $X(0) = 0$ e $X(L) = 0$ implica que a equação em x tem solução não identicamente nula somente se $\lambda < 0$ sendo λ igual a $\sqrt{-\lambda} L = \pi n \Rightarrow \sqrt{-\lambda} = \frac{\pi n}{L} \Rightarrow$

$\lambda = -\frac{\pi^2 n^2}{L^2}$, para $n=1,2,3,\dots$. Substituindo λ em $X(x)$ obtém $X(x) = c_1 \cdot \text{sen}(\frac{\pi n x}{L})$, dessa forma $X(x)$ gera “n” soluções fundamentais: $X_n(x) = \text{sen}(\frac{\pi n x}{L})$.

Substituindo $\lambda = -\frac{\pi^2 n^2}{L^2}$ na segunda linha da Figura 6, tem que $T'(t) + \alpha \frac{\pi^2 n^2}{L^2} T(t) = 0$, uma equação diferencial do formato $\frac{dy}{dx} + P(x) \cdot y = Q(x)$, com solução fundamental $T_n(t) = e^{-\frac{\alpha \pi^2 n^2}{L^2} t}$, para $n = 1, 2, 3, \dots$

Finalmente $w_n(x,t) = X_n(x) \cdot T_n(t) = \text{sen}(\frac{\pi n x}{L}) \cdot e^{-\frac{\alpha \pi^2 n^2}{L^2} t}$, $n = 1, 2, 3, \dots$

Combinações lineares das soluções fundamentais são também solução, $w(x,t) = \sum_{n=1}^N c_n w_n(x,t) = \sum_{n=1}^N c_n \text{sen}(\frac{\pi n x}{L}) \cdot e^{-\frac{\alpha \pi^2 n^2}{L^2} t}$, mas uma solução assim não necessariamente satisfaz a condição inicial $w(x,0) = f(x) - v(x)$.

É necessário supor então que a solução do problema possa ser escrito como uma série, com N tendendo ao infinito $w(x,t) = \sum_{n=1}^{\infty} c_n \text{sen}(\frac{\pi n x}{L}) \cdot e^{-\frac{\alpha \pi^2 n^2}{L^2} t}$, considerando que será necessário utilizar o valor inicial da EDP homogênea, que por sua vez é $w(x,0) = f(x) - v(x)$ então $f(x) - v(x) = \sum_{n=1}^{\infty} c_n \text{sen}(\frac{\pi n x}{L}) = w(x,0)$.

$v, f: [0, L] \rightarrow \mathbb{R}$ é contínua por partes e $f'(x)$ e $v'(x)$ também são, então $f(x) - v(x) = \sum_{n=1}^{\infty} c_n \text{sen}(\frac{\pi n x}{L})$.

$\sum_{n=1}^{\infty} c_n \text{sen}(\frac{\pi n x}{L})$ é a série de Fourier em senos e os coeficientes são dados por $c_n = \frac{2}{L} \int_0^L (f(x) - v(x)) \text{sen}(\frac{\pi n x}{L}) dx$, $n = 1, 2, 3, \dots$, substituindo os valores que são dados: $c_n = 2 \int_0^L -100x \text{sen}(\pi n x) dx$ e calculando a integral, $c_n = \frac{-200(\text{sen}(\pi n) - \pi n \cos(\pi n))}{\pi^2 n^2}$. $\text{Sen}(\pi n)$ é 0 para todo n e $\cos(\pi n) = (-1)^n$, pois varia entre -1 a 1, simplificando, $c_n = \frac{-200(0 - \pi n (-1)^n)}{\pi^2 n^2} = \frac{-200(-\pi n (-1)^n)}{\pi^2 n^2} = \frac{200(-1)^n}{\pi n}$.

Assim, a solução numérica da equação de difusão de calor unidimensional é dada por $w(x,t) = v(x) + \sum_{n=1}^{\infty} c_n w_n(x,t) = v(x) + \sum_{n=1}^{\infty} c_n \text{sen}(\frac{\pi n x}{L}) \cdot e^{-\frac{\alpha \pi^2 n^2}{L^2} t}$, substituindo os valores que temos então, $w(x,t) = 100x + \sum_{n=1}^{\infty} \frac{200(-1)^n}{\pi n} \text{sen}(\pi n x) \cdot e^{-\alpha \pi^2 n^2 t}$.

$$T(x, t) = 100x + \sum_{n=1}^{\infty} (-1)^n \frac{200}{n\pi} \text{sen}(n\pi x) e^{-\alpha t (n\pi)^2}.$$

Para construir os gráficos foi usado o programa Matlab ,escolhendo um α e t arbitrários e para cada ponto de x , é calculado o $T(x)$ e plotando uma coordenada $(x, T(x))$ no gráfico.

Para $\alpha = 1.304348$ e $t = 0.01$, $T(x) = 100x +$

$\sum_{n=1}^{\infty} \frac{200(-1)^n}{n\pi} \text{sen}(\pi n x) \cdot e^{-1.204348\pi^2 n^2 0.01}$, foi obtido o seguinte resultado.

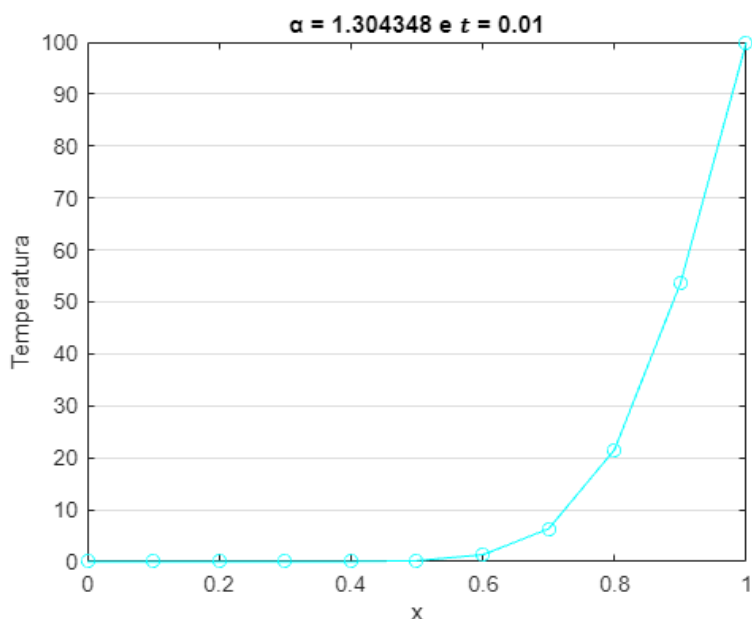


Figura 4 - Resultados obtidos para $\alpha = 1.304348$ e $t = 0.01$.

Foi utilizada o keras e tensor flow para construir e implementar a rede neural, no ambiente do google colab, que é uma plataforma online gratuita do Google que permite a execução de código Python no navegador, ela foi escolhida pois existe um acesso gratuito a GPUs, para que o tensor flow possa funcionar corretamente.

Para a amostragem da solução exata, a função foi dividida em 11 pontos e uma amostra foi coletada a cada ponto dessa função, além disso, foram criados os dados para calcular a EDP, com 20 pontos em x de 0 a 1.

Após isso, foi definida a arquitetura do modelo neural, 1 neurônio para camada de saída, contendo os valores da amostragem, 1 neurônio para a camada de saída, que compara o dado calculado em relação aos dados da amostragem, e 2 camadas de rede densa com 32 neurônios cada. São definidos os hiperparâmetros, como os pesos através

do tensorflow, a taxa de aprendizado igual a 0.0005 e 30.000 épocas para realizar o aprendizado.

Com o keras é construído a rede neural, onde foi adicionado um bias em cada camada, função de ativação tangente hiperbólica para os neurônios da camada oculta e a função de ativação linear para a camada de saída.

Foi criado várias variáveis que armazenam informações da rede neural a cada época para que essas informações sejam plotadas posteriormente.

Durante o treinamento, é calculado o custo da rede física (MSEfísico) e o custo dos dados de entrada (MSEdados), esses dois custos são somados para se ter o custo total, então o algoritmo de aprendizado vai procurar pesos e bias que irão encontrar o menor custo total e os gradientes da rede neural são utilizados para a função de otimização ADAM.

$$\text{MSE}_{\text{dados}} = \frac{1}{n} \sum_{i=1}^m (\hat{y} - y)^2 \text{ e } \text{MSE}_{\text{físico}} = \frac{1}{n} \sum_{i=1}^m \left(\alpha \cdot \frac{\partial^2 T(x)}{\partial^2 x} \right)^2.$$

Considerando o $\alpha = 1.304348$ e $t = 0.01$, temos os resultados numéricos apresentados a seguir.

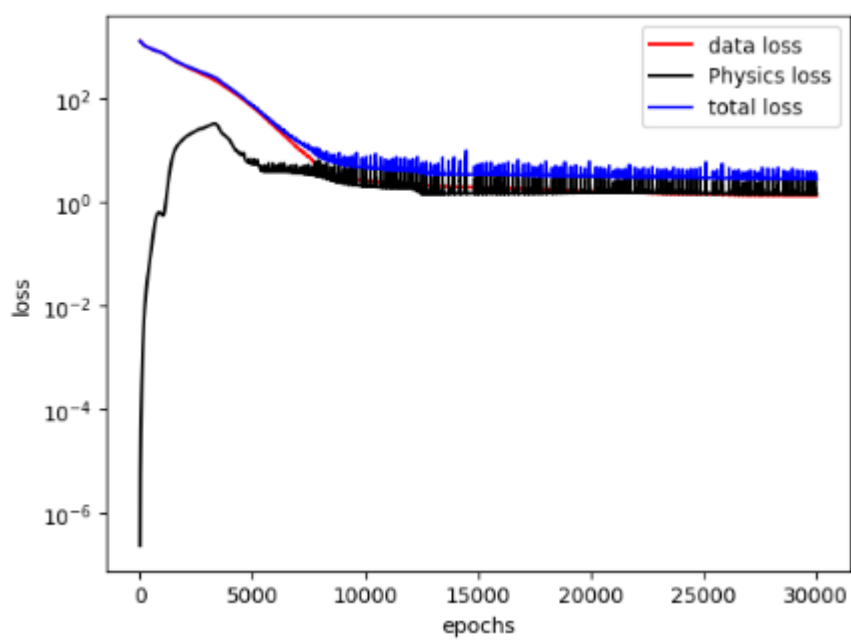


Figura 5 - Custos durante as épocas.

O custo vai diminuindo durante as épocas, significa que o programa está funcionando corretamente, a partir da época 7500, a função começa a estagnar, então

15000 já seriam suficientes para obter um resultado satisfatório.

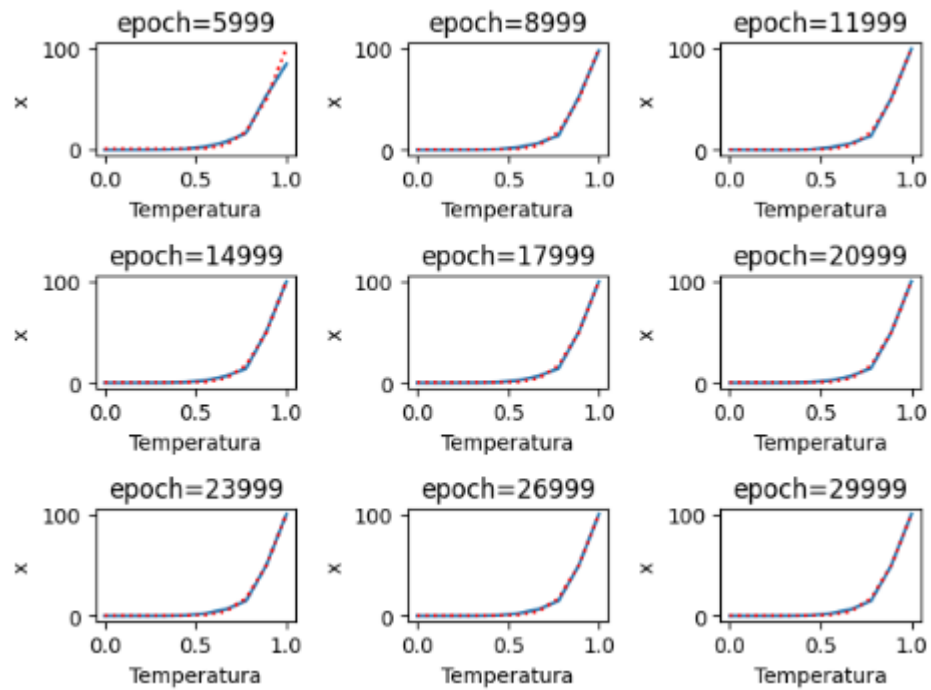


Figura 6 - Aproximação da predição ao longo das épocas.

Considerando agora $\alpha = 1.304348$ e $t = 0.01$, temos:

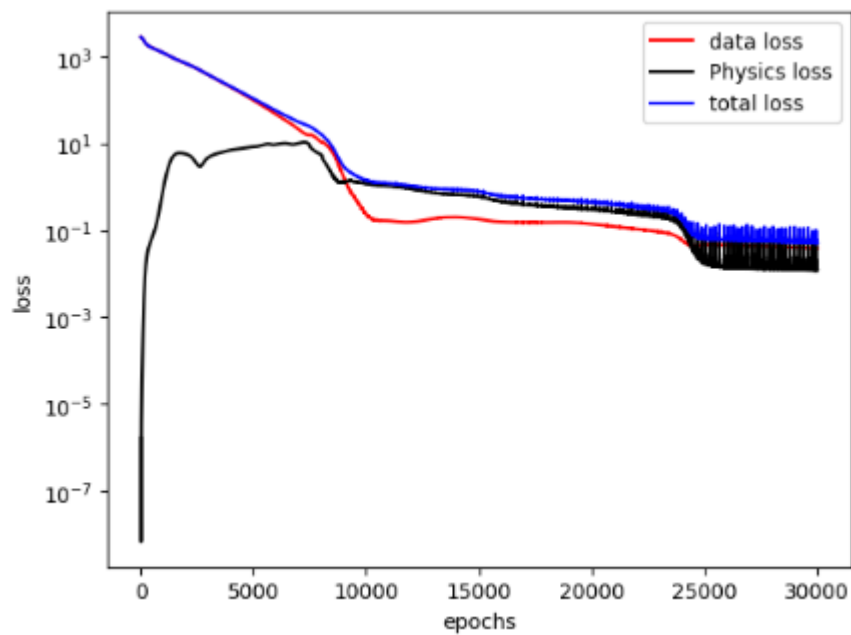


Figura 7 - Custos durante as épocas.

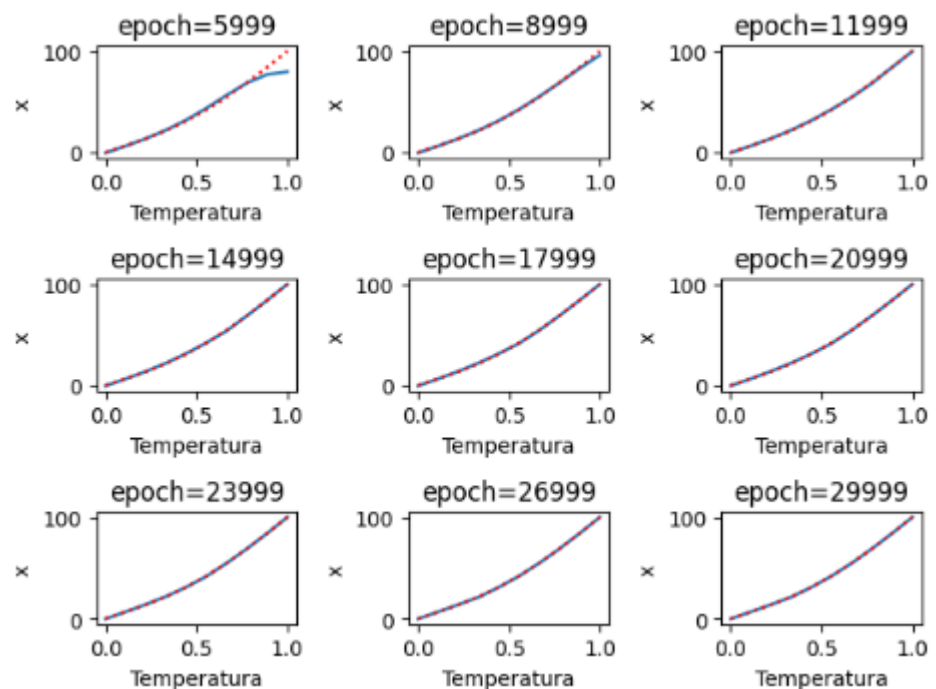


Figura 8 - Aproximação da predição ao longo das épocas.

Os erros absolutos são calculados através da diferença entre a solução analítica e a predição da rede neural em cada ponto mostrado nos gráficos. São 11 pontos divididos entre 0 e 1 do eixo-x, que por sua vez são os pontos espaçados igualmente da barra.

À cada dois pontos é traçado uma linha, formando o gráfico inteiro através de

todos os pontos, isso mostra que poderiam ter sido traçados mais pontos, dividindo a malha em intervalos menores, fazendo com que a aproximação seja mais precisa, diminuindo o erro absoluto, porém, para fins deste estudo, a divisão já é satisfatória, mostrando que o uso das redes neurais para a solução de cálculos numéricos é de boa aproximação.

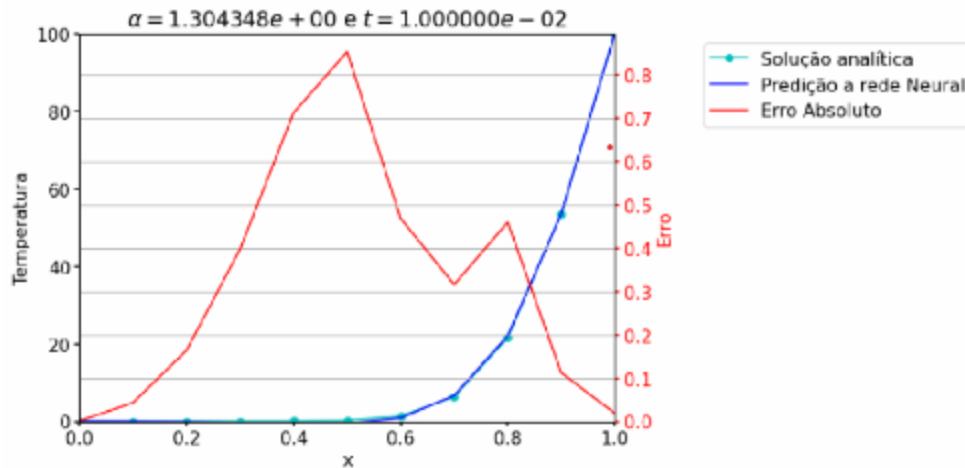


Figura 9 - Resultados obtidos para $\alpha = 1.304348$ e $t = 0.01$.

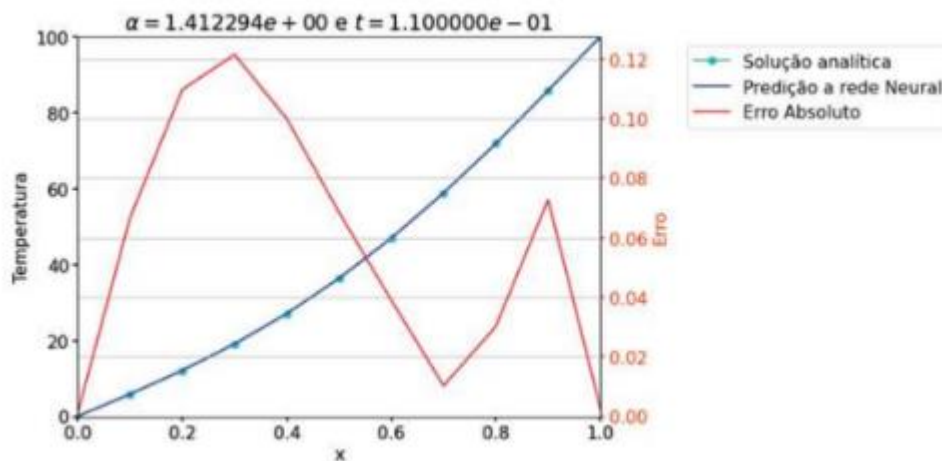


Figura 10 - Resultados obtidos para $\alpha = 1.412294$ e $t = 0.11$.

É possível ver que o primeiro ponto $T_{init} = 0\text{ }^{\circ}\text{C}$ e o último ponto $T_{fim} = 100\text{ }^{\circ}\text{C}$ condizem com as temperaturas que cada extremidade ($x = 0\text{m}$ e $x = 1\text{m}$) foram submetidas.

O gráfico mostra qual é a temperatura que cada ponto da barra possui quando se passa um tempo específico.

Parece que a solução analítica e a predição da rede neural têm o mesmo valor, porém se um zoom for dado nos gráficos onde o erro absoluto tem um pico, é possível perceber que a solução analítica e a predição da rede neural tem uma pequena diferença. O valor máximo que o erro absoluto teve, dados os gráficos apresentados, foi de 0.12, valores que condizem com um erro aproximado de primeira ordem.

Dado que a barra foi dividida em 11 pontos igualmente espaçados e que o erro absoluto máximo entre os dois gráficos foi de 1.12, é possível mostrar que o uso das redes neurais para a solução de cálculos numéricos permite realizar boas aproximações.

4. CONCLUSÕES

O uso das redes neurais foi eficiente na solução de problemas numéricos, como na solução da equação de difusão de calor unidimensional, tanto para valores de coeficiente de difusividade térmica e tempos distintos, apresentando bons resultados e bom desempenho para obter respostas, com um erro aproximado de primeira ordem, assim portanto, mostrando que o uso de redes neurais podem também ser uma forma de apresentar boas aproximações para modelos numéricos do cálculo científico.

Os resultados obtidos demonstraram que a abordagem baseada em PINNs foi capaz de fornecer boas aproximações para a solução da equação de difusão de calor, apresentando um desempenho satisfatório em termos de precisão e tempo de convergência. Com isso, este estudo reforça o potencial das redes neurais na solução de problemas numéricos em cálculo científico, oferecendo uma alternativa viável e eficiente aos métodos numéricos tradicionais.

5. AGRADECIMENTOS

Agradecemos ao Programa Institucional de Bolsas de Iniciação Científica (PIBIC)/Reitoria pelo auxílio financeiro no desenvolvimento deste projeto.

6. REFERÊNCIAS

[1] DAS, S. et al. **Applications of artificial intelligence in machine learning: Review**

and prospect. International Journal of Computer Applications, v. 115, p. 31–41, 201

[2] FACELI, K. et al. **Inteligência Artificial: uma abordagem de aprendizado de máquina.** Rio de Janeiro: LTC, 2011.

[3] FORTUNA, A. O. **Técnicas computacionais para dinâmica dos fluidos:** conceitos básicos e aplicações. 2.ed. São Paulo: EDUSP, 2012.

[4] MALISKA, C.R. (1995). **Transferência de calor e mecânica dos fluidos computacional,** Rio de Janeiro: Livros Técnicos e Científicos.

[5] MOHRI, M.; ROSTAMIZADEH, A.; TALWALKAR, A. **Foundations of Machine Learning.** [S.l.]: The MIT Press, 2012.

[6] TZANIS, G. et al. **Modern applications of machine learning.** Proceedings of the 1st Annual SEERC Doctoral Student Conference. 2006.